BDIO
BIG DATA & IOT

ENSAM
ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS
UNIVERSITÉ HASSAN II DE CASABLANCA

DevOps & MLOps Project

End-to-End MLOps Workflow

CI/CD, Dockerized Training, Model Serving, and Monitoring

Text Classification System using Jenkins, Ray, MLflow, FastAPI, Prometheus, and Grafana

Prepared by:

Omar Elkhali

Ayoub Moutik

Jamila Ouzzane

Supervised by:

Prof. HIRCHOUA Bader

Academic Year: 2025–2026

Contents

Abstract

This report presents the design and implementation of an MLOps workflow for a machine learning text classification project. The system is organized around a modular Python codebase that supports data processing, model training, evaluation, prediction, serving, CI/CD automation, and monitoring. The workflow emphasizes maintainability, traceability, reproducibility, and operational visibility, which are essential requirements for reliable machine learning systems.

The report begins by describing the transition from notebook-based experimentation to script-based engineering. It then presents the logging and reproducibility mechanisms provided by Git, dependency management, MLflow tracking, model checkpointing, dataset organization, and deterministic utilities. The final sections explain the Dockerized Jenkins CI/CD pipeline, the deployment gate based on model quality, and the local monitoring stack built with FastAPI, Prometheus, and Grafana.

1 Project Context And Objective

1.1 Context

The project material is composed of five course PDFs and two repositories:

- `MLOpsFull`: the main machine learning application.
- `Monitoring-ML`: a monitoring notebook and README covering model performance, drift detection, and online monitoring.

The main application is a text classification system inspired by the Made With ML project. It classifies machine learning project descriptions into tags such as computer vision, natural language processing, graph learning, MLOps, reinforcement learning, and other categories.

1.2 Objective

The objective is to demonstrate an end-to-end MLOps workflow. The project includes the foundational engineering practices required before automation and monitoring can be introduced reliably:

1. moving from notebooks to scripts;
2. logging for ML systems;
3. reproducibility through versioned code, data, models, and dependencies.

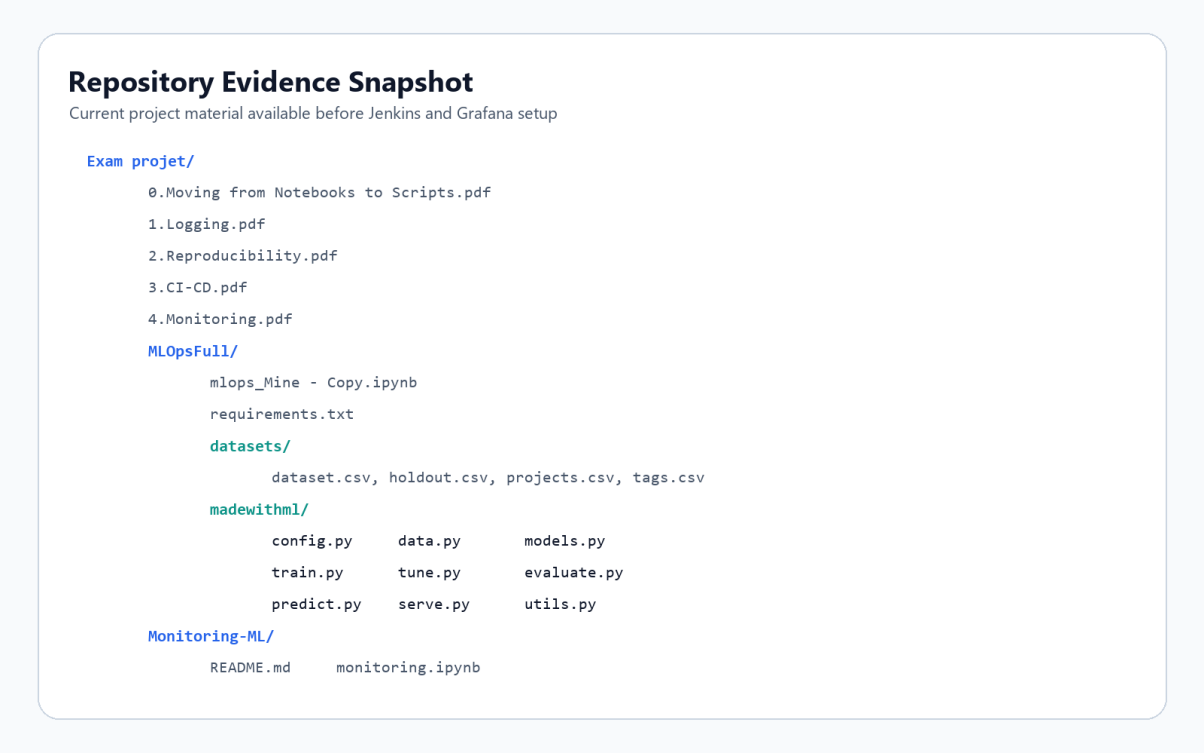

Figure 1: Current project material available in the workspace.

2 Building The MLOps Workflow

2.1 Part 1: Moving From Notebooks To Scripts

2.1.1 Purpose

Notebooks are excellent for exploration because they allow interactive execution, visual inspection, and fast experimentation. However, they are not ideal as the final structure of a production ML project. A notebook can hide execution order, retain variables in memory, and make testing or automation difficult.

Scripts provide a cleaner engineering structure. They make workloads easier to test, execute from a terminal, reuse in CI/CD, and deploy as services. This is the first major step from data science experimentation toward MLOps.

2.1.2 Existing Notebook

The original notebook is:

```
MLOpsFull/mlops_Mine - Copy.ipynb
```

Its main sections include setup, data ingestion, preprocessing, training, experiment tracking, evaluation, interpretability, prediction, and serving. This is a typical research-to-development notebook: it contains the full workflow, but the logic is not yet presented as independent production components.

2.1.3 Script-Based Architecture

The notebook workflow has been separated into modules under `MLOpsFull/madewithml`. Each file owns a specific responsibility, which improves maintainability and makes the project easier to automate.

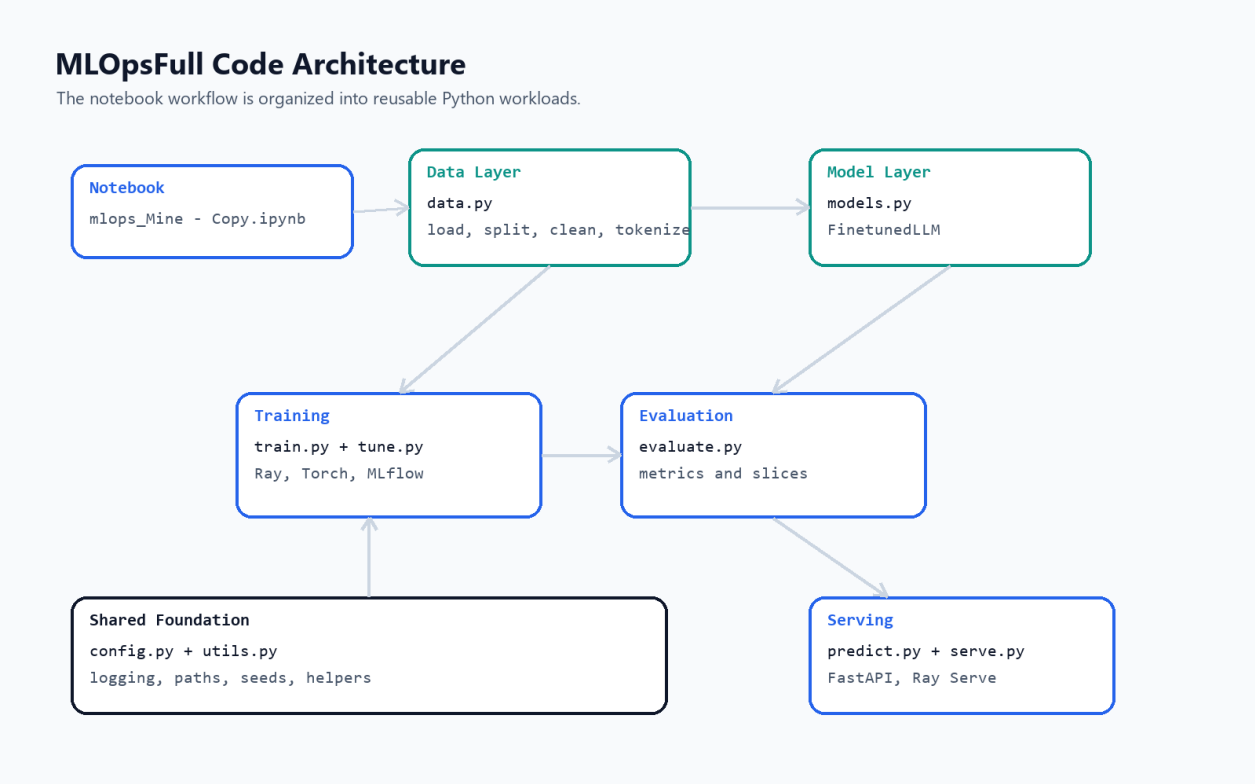

Figure 2: Script organization of the MLOpsFull project.

2.1.4 Module Responsibilities

Module	Responsibility
`config.py`	Defines shared project configuration, logging paths, MLflow tracking URI, and constants such as stopwords.
`data.py`	Loads datasets, performs stratified splitting, cleans text, tokenizes input, and applies preprocessing.
`models.py`	Defines the PyTorch model architecture used for text classification.
`train.py`	Contains the Ray Train workload, training loop, checkpointing, and MLflow callback integration.
`tune.py`	Contains the hyperparameter tuning workload using Ray Tune and HyperOpt.
`evaluate.py`	Computes overall, per-class, and slice-level metrics.
`predict.py`	Loads model checkpoints and performs prediction or probability estimation.
`serve.py`	Defines the FastAPI application and Ray Serve deployment.
`utils.py`	Provides shared utility functions such as seed setting, batching, and dictionary saving.

2.1.5 Notebook-To-Script Mapping

Notebook Area	Script Destination
Data loading and preprocessing	`madewithml/data.py`
Model definition	`madewithml/models.py`
Training loop	`madewithml/train.py`
Hyperparameter search	`madewithml/tune.py`
Evaluation	`madewithml/evaluate.py`
Prediction	`madewithml/predict.py`
Serving	`madewithml/serve.py`
Shared configuration	`madewithml/config.py`
Shared helpers	`madewithml/utils.py`

Table 2: Mapping from notebook sections to production-oriented scripts.

2.1.6 Benefits For MLOps

This structure is important because Jenkins, tests, model training jobs, and deployment commands all need scriptable entry points. A CI/CD server cannot reliably execute arbitrary notebook cells in a human-controlled order. It needs deterministic commands and modules.

The current organization creates a foundation for:

- command-line training;
- automated evaluation;
- experiment tracking;
- API deployment;
- future Jenkins automation.

2.2 Part 2: Logging For ML Systems

2.2.1 Purpose

Logging records what happens inside the application during execution. In ML systems, logging is essential because failures can come from code, data, training configuration, model artifacts, infrastructure, or serving inputs.

Compared with `print`, logging is more suitable because it supports severity levels, multiple outputs, timestamps, persistent files, and consistent formatting.

2.2.2 Logging Configuration In The Project

The logging setup is defined in:

```
MLOpsFull/madewithml/config.py
```

The project creates a `logs` directory and defines several handlers:

- a console handler for terminal output;
- an `info.log` rotating file handler for informative logs;
- an `error.log` rotating file handler for errors.

Key implementation pattern:

```
LOGS_DIR = Path(ROOT_DIR, "logs")
LOGS_DIR.mkdir(parents=True, exist_ok=True)

logging_config = {
    "version": 1,
    "disable_existing_loggers": False,
    "handlers": {
        "console": {...},
        "info": {...},
        "error": {...},
    },
    "root": {
        "handlers": ["console", "info", "error"],
        "level": logging.INFO,
    },
}

logging.config.dictConfig(logging_config)
logger = logging.getLogger()
```

2.2.3 Where Logs Are Used

The project uses `logger.info(...)` to record important workflow outputs. For example:

- training logs the timestamp, run ID, parameters, and metric history;
- evaluation logs overall metrics, per-class metrics, and slice metrics;
- prediction logs model prediction results.

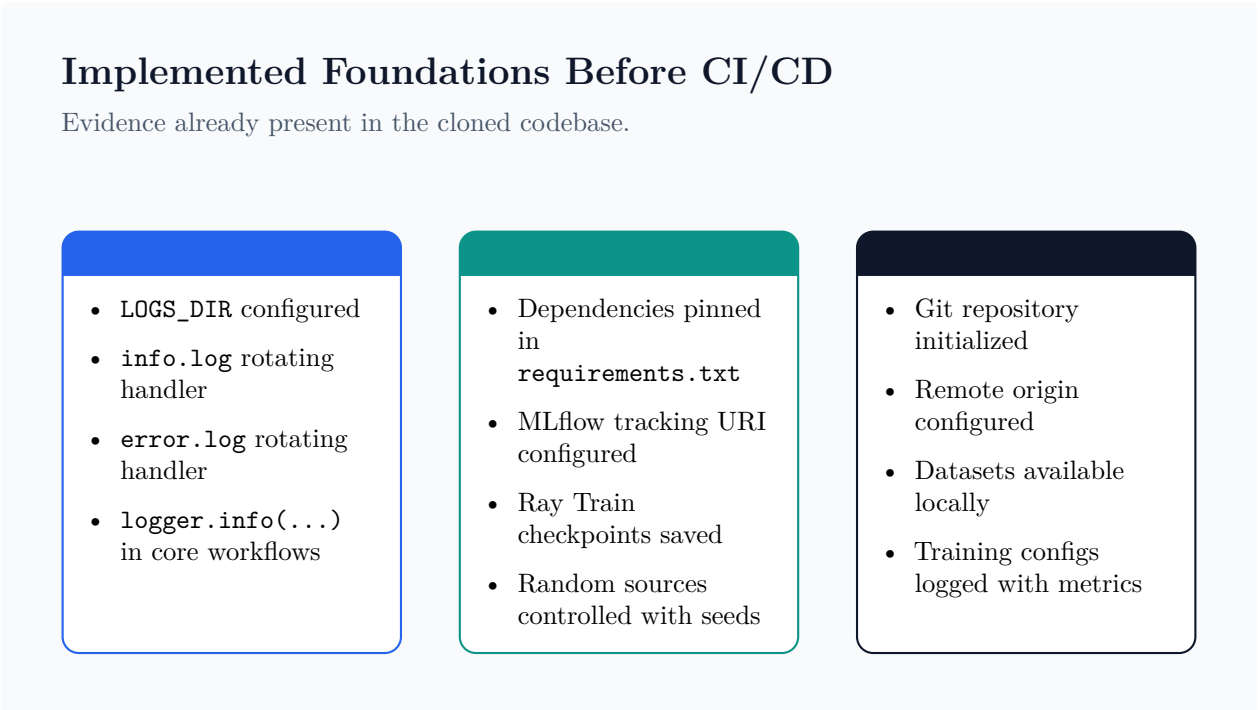

Figure 3: Evidence of implemented logging, reproducibility, and traceability foundations.

2.2.4 Why Logging Matters For This Project

In this project, logging supports three important MLOps needs:

1. **Debugging**: errors and warnings can be inspected after a failed run.
2. **Traceability**: training and evaluation outputs can be connected to model runs.
3. **Monitoring readiness**: logs can later be exported to tools such as Loki, Elasticsearch, or cloud logging services and visualized in Grafana.

2.2.5 Production Extension

For the final version, logs can be connected to the monitoring stack. A practical production-style design would be:

```
Application logs -> Loki or Elasticsearch -> Grafana dashboards
```

This will allow the report to include screenshots of API errors, model prediction logs, and system events once the serving and monitoring stages are implemented.

2.3 Part 3: Reproducibility

2.3.1 Purpose

Reproducibility means being able to recreate a previous result. In machine learning, this is difficult because results depend on code, data, dependencies, random seeds, training parameters, model artifacts, and hardware.

A reproducible ML project should answer:

- Which code version produced this model?
- Which dataset was used?
- Which dependencies were installed?
- Which parameters were used?
- Which metrics were obtained?
- Where is the model artifact stored?

2.3.2 Code Versioning With Git

The repositories are Git repositories and are connected to a remote origin. Git provides history, collaboration, rollback, and traceability.

```
origin https://github.com/AyoubMoutik/mlops-project.git
```

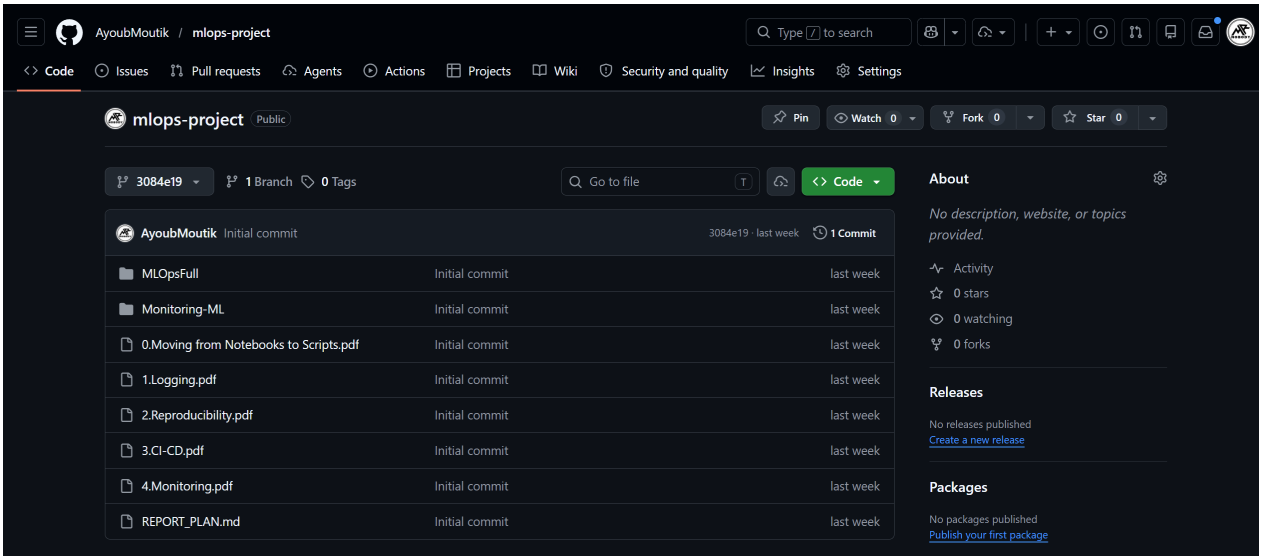

Figure 4: GitHub repository evidence showing the project remote, tracked files, and initial commit history.

2.3.3 Dependency Versioning

The project includes:

```
MLOpsFull/requirements.txt
```

This file pins or constrains many important dependencies, including:

- `mlflow`
- `ray`
- `torch`
- `transformers`
- `scikit-learn`
- `fastapi`
- `pytest`
- `great-expectations`

This is necessary because dependency changes can alter model behavior or break the pipeline.

2.3.4 Data Versioning

For this educational project, the datasets are stored directly in the repository:

```
MLOpsFull/datasets/dataset.csv
MLOpsFull/datasets/holdout.csv
MLOpsFull/datasets/projects.csv
MLOpsFull/datasets/tags.csv
```

This is acceptable because the datasets are small. In production, larger datasets should not usually be stored directly in Git. Better options include:

- DVC;
- Git LFS;
- S3 or another object store;

- Snowflake or another warehouse;
- versioned data lake storage.

2.3.5 Model And Experiment Versioning With MLflow

The project configures MLflow tracking in `config.py`. The model registry path is derived from the configured storage directory, and MLflow is set using a file URI.

```
MODEL_REGISTRY = Path(f"{EFS_DIR}/mlflow")
MLFLOW_TRACKING_URI = "file://" + str(MODEL_REGISTRY.absolute())
mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)
```

During training, the project uses an MLflow callback:

```
mlflow_callback = MLflowLoggerCallback(
    tracking_uri=MLFLOW_TRACKING_URI,
    experiment_name=experiment_name,
    save_artifact=True,
)
```

This supports experiment tracking by storing parameters, metrics, checkpoints, and artifacts.

2.3.6 Checkpointing

The training workflow creates checkpoints and keeps the best checkpoint based on validation loss:

```
checkpoint_config = CheckpointConfig(
    num_to_keep=1,
    checkpoint_score_attribute="val_loss",
    checkpoint_score_order="min",
)
```

This matters because the best model can be loaded later for prediction, evaluation, or deployment.

2.3.7 Random Seeds

The project contains a utility function for setting random seeds. This reduces randomness across NumPy, Python, PyTorch, CUDA, and hashing behavior.

```
def set_seeds(seed: int = 42):
    np.random.seed(seed)
    random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)
```

This does not guarantee perfect reproducibility on every machine, but it is an important baseline.

2.3.8 Reproducibility Summary

Requirement	Current Project Evidence
Code versioning	Git repository with remote origin configured.
Dependency versioning	`requirements.txt`.
Data availability	CSV datasets in `MLOpsFull/datasets`.
Experiment tracking	MLflow tracking URI and Ray MLflow callback.
Model artifact tracking	Ray checkpoints and MLflow artifacts.
Configuration tracking	Training parameters logged with results.
Randomness control	`set_seeds` utility.

Table 3: Current reproducibility mechanisms in the project.

2.4 Part 4: CI/CD With Jenkins

2.4.1 Course Concept

The CI/CD course material explains that machine learning systems should not depend on manual execution. Instead, workflows should be triggered by events such as pull requests, pushes to the main branch, new data, scheduled intervals, or detected performance regressions. The goal is to create a system that can continually improve while still giving the team control over when a model is promoted.

The ideal course workflow is:

```
change code -> pull request -> model workloads -> quality check
          -> merge to main -> deployment workflow -> production service
```

In this project, this idea is implemented with Jenkins and Docker. The pipeline does not only test the code. It builds a reproducible image, trains a model, evaluates it, pushes the image to Docker Hub, deploys the API, and runs smoke checks.

2.4.2 Docker-First CI/CD Architecture

The final implementation uses a Docker-first strategy. Jenkins itself runs in Docker, but it is connected to the host Docker daemon so it can build application images, create short-lived workload containers, push images, and deploy the model service.

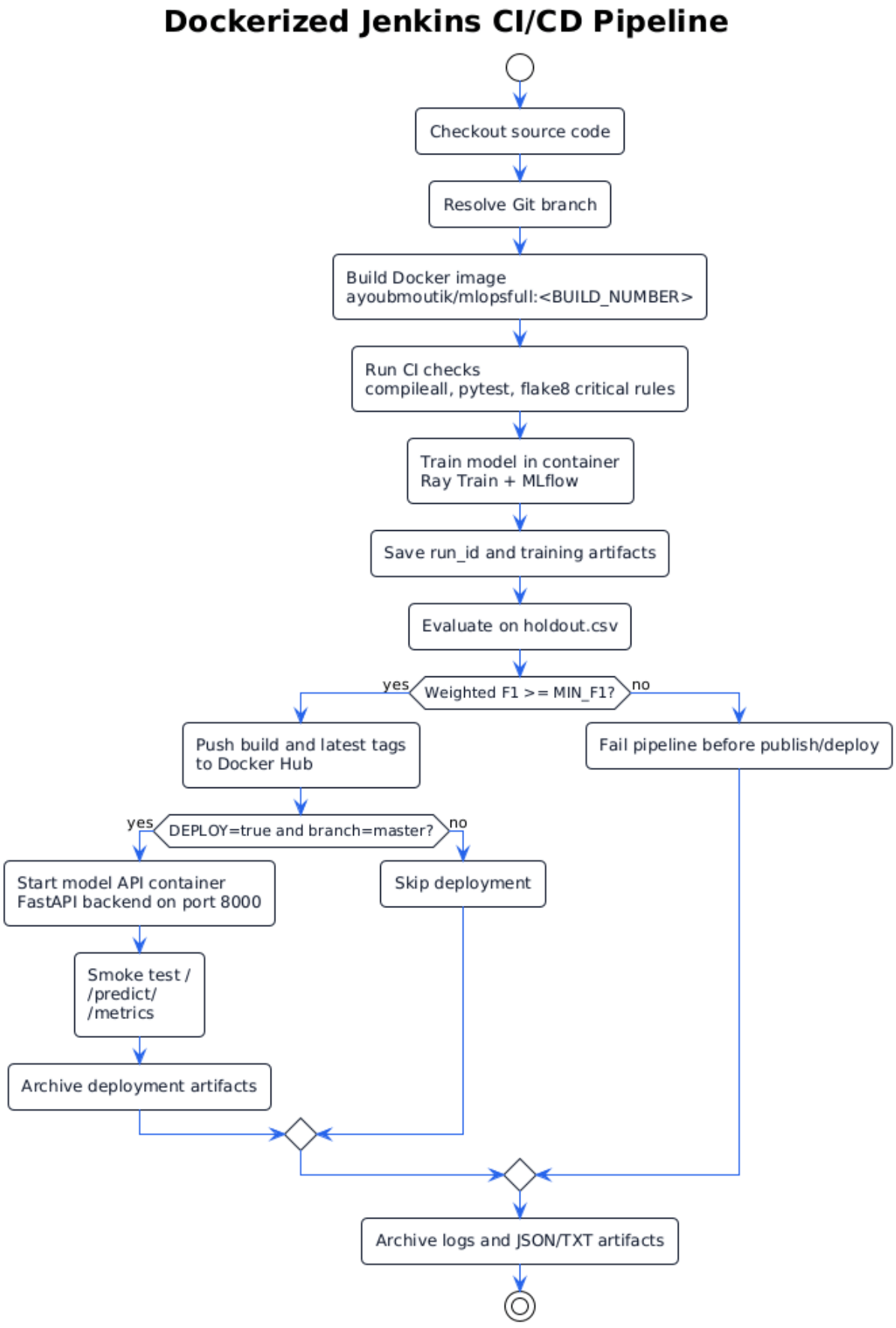

Figure 5: PlantUML diagram of the Dockerized Jenkins CI/CD pipeline.

The corresponding PlantUML source is stored in:

```
report/diagrams/cicd_pipeline.puml
```

2.4.3 Files Added For CI/CD

File	Purpose
`Jenkinsfile`	Defines the CI/CD stages: checkout, branch resolution, Docker build, checks, training, evaluation, Docker Hub push, deployment, and artifact archiving.
`Dockerfile.jenkins`	Builds a Jenkins controller image with Docker CLI support.
`docker-compose.yml`	Starts Jenkins and later Prometheus/Grafana; mounts the Docker socket for Docker control.
`MLOpsFull/Dockerfile`	Builds the Python 3.10 ML application image.
`MLOpsFull/.dockerignore`	Keeps notebooks, caches, logs, artifacts, and local files out of Docker builds.

2.4.4 Jenkins Pipeline Stages

The implemented pipeline contains these stages:

1. **Checkout**: pulls the project source code from Git.
2. **Resolve Branch**: normalizes branch names such as `origin/master` into `master`.
3. **Build Docker Image**: builds the versioned application image and the `latest` tag.
4. **CI Checks**: runs Python compilation, tests, and critical Flake8 rules inside the image.
5. **Train**: trains the model in a container using Ray Train and MLflow.
6. **Evaluate**: evaluates the trained run on the holdout dataset.
7. **Push Image to Docker Hub**: publishes the image only after training and evaluation pass.
8. **Deploy**: deploys the service only when `DEPLOY=true` and the branch is `master`.
9. **Post Actions**: archives JSON, TXT, and LOG artifacts for evidence.

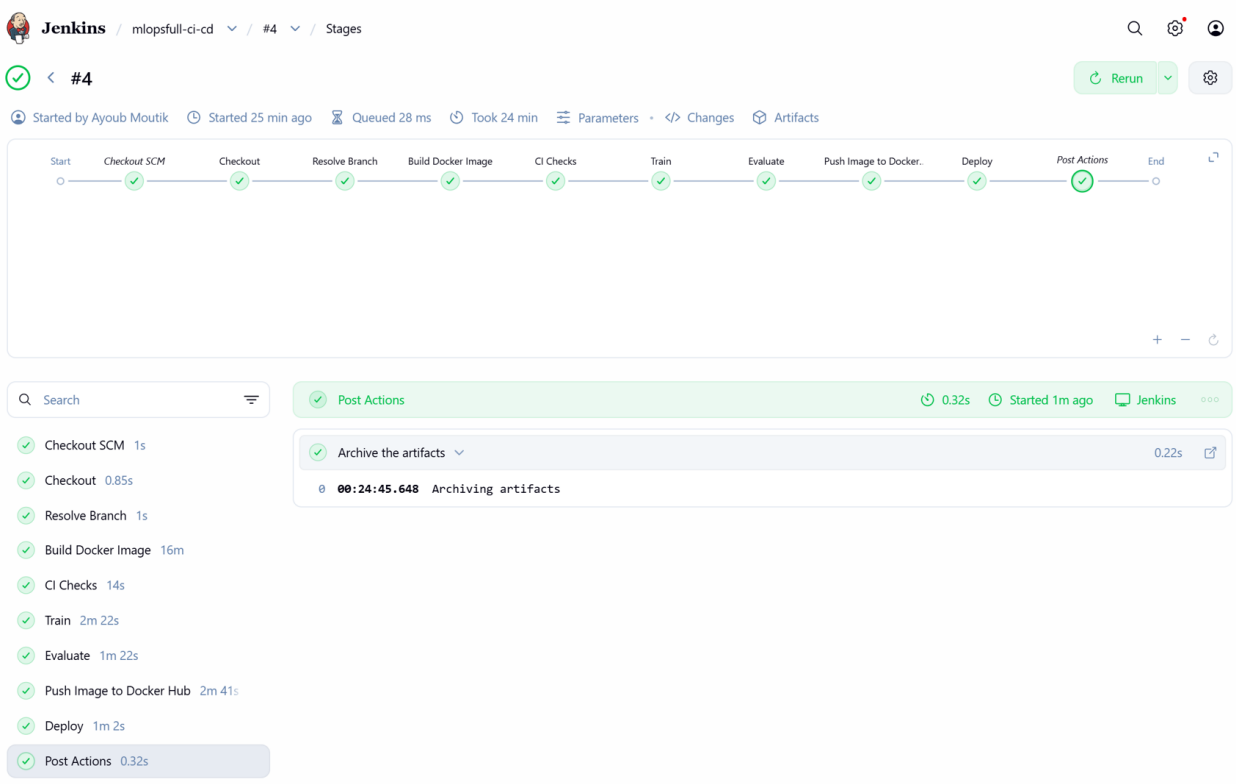

Figure 6: Jenkins build evidence showing all CI/CD stages completed successfully.

2.4.5 Pipeline Parameters

The Jenkins job is configurable so that the same pipeline can be used for fast demonstrations or heavier training runs.

Parameter	Default	Purpose
`NUM_SAMPLES`	`0`	Uses the full dataset for the Jenkins training run.
`NUM_EPOCHS`	`6`	Controls training epochs.
`BATCH_SIZE`	`16`	Controls model training batch size.
`MIN_F1`	`0.85`	Minimum weighted F1 score required before deployment can continue.
`USE_GPU`	`true`	Enables NVIDIA GPU acceleration for the training container when available.
`DEPLOY`	`true`	Enables or disables local deployment after evaluation.

Table 5: Jenkins parameters used to control the CI/CD run.

2.4.6 Quality Gates

The pipeline contains two important quality gates. The first one is the CI gate:

```
python -m compileall madewithml
python -m pytest tests -q
python -m flake8 madewithml --select=E9,F63,F7,F82
```

This checks for syntax errors, runs the test suite, and limits Flake8 to critical categories that can break execution.

The second gate is the model evaluation gate. The Jenkinsfile reads the weighted F1 score from `eval_results.json` and compares it with `MIN_F1`. If the model fails the threshold, the pipeline stops before Docker Hub publishing and deployment.

Model Quality Result

The successful Dockerized Jenkins run produced a weighted F1 of `0.9054760519681985`. The model passed the configured `MIN_F1=0.85` deployment gate and was promoted to the local serving stage.

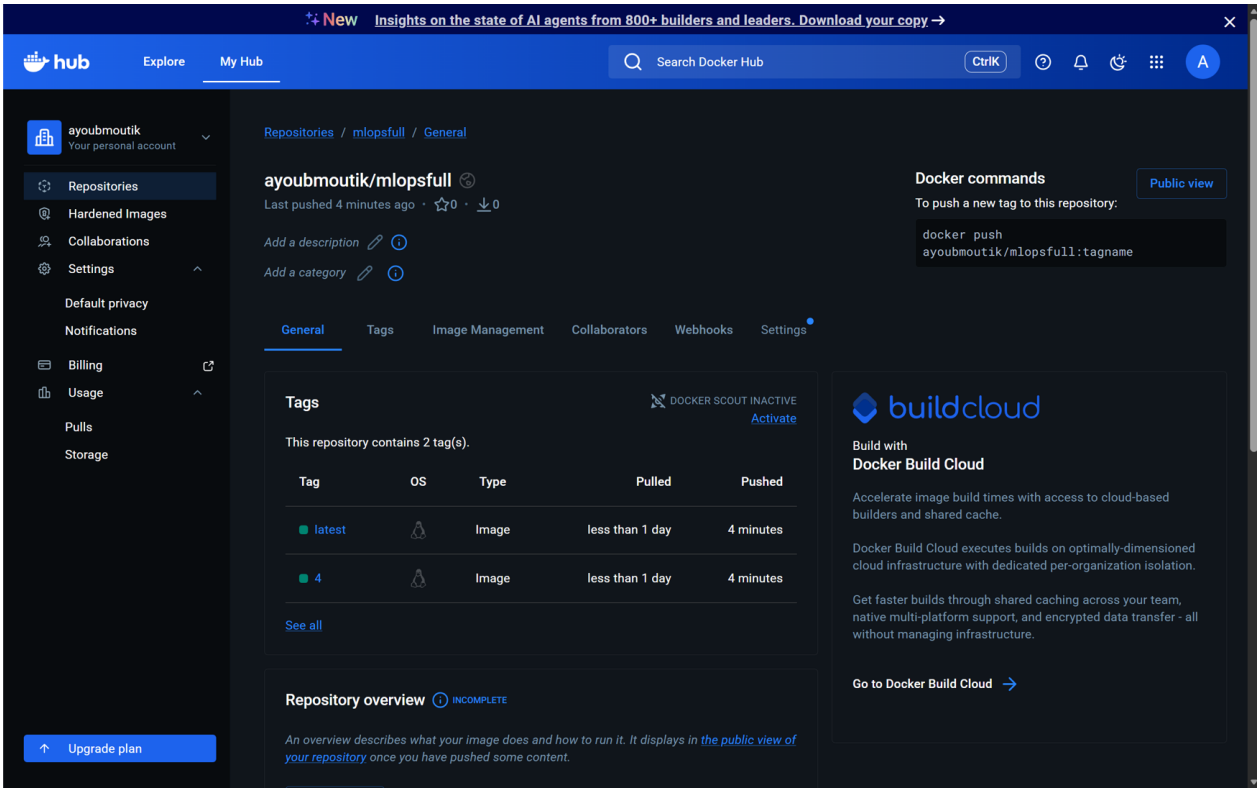

Figure 7: Docker Hub evidence showing the pushed `ayoubmoutik/mlopsfull` image tags.

2.4.7 Containerized Stage Execution

The helper function `dockerRun(...)` makes the CI, training, and evaluation stages run inside the built application image. Jenkins writes a temporary shell script, creates a temporary container, copies the script into the container, starts it, copies artifacts back from `/app/artifacts`, and removes the container.

This gives the pipeline one very important property: the checks and ML workloads run in the same type of environment that will later serve the model.

2.4.8 Deployment And Smoke Tests

The deployment stage starts the model service container:

```
python -m madewithml.serve --run_id "$RUN_ID" --backend fastapi
```

The service exposes port `8000`. Jenkins validates deployment using smoke tests:

- `GET /`: confirms that the service is reachable.

- `POST /predict/`: confirms that the model can return a prediction.
- `GET /metrics`: confirms that Prometheus metrics are exposed.

Smoke tests do not prove full correctness, but they catch the most important deployment failures: container crash, API startup failure, model loading failure, prediction failure, and missing metrics.

2.4.9 CI/CD Evidence

Evidence	Meaning
`train_results.json`	Training timestamp, run ID, parameters, and metric history.
`run_id.txt`	MLflow run ID passed from training to evaluation and deployment.
`eval_results.json`	Overall, per-class, and slice-level evaluation metrics.
`deploy_health.json`	Response from the deployed API health endpoint.
`smoke_response.json`	Prediction response from the deployment smoke test.
`metrics_smoke.txt`	Prometheus-format metrics returned by the deployed API.
`deploy_container.log`	Container logs captured when deployment fails.

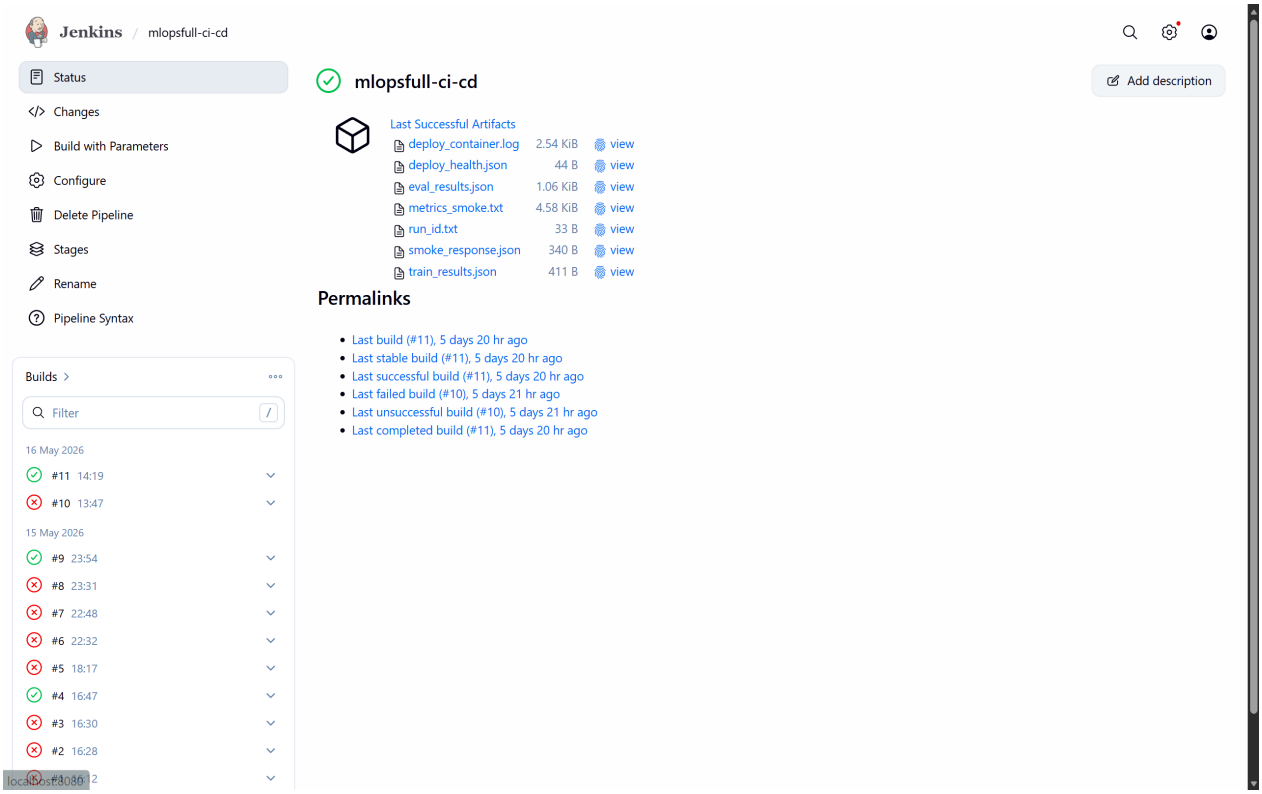

Figure 8: Jenkins archived artifacts evidence showing training, evaluation, deployment health, smoke-test, metrics, and log outputs.

2.4.10 Model Serving

Serving Backend Decision The original project supports Ray Serve, and Ray remains central to the project for Ray Data, Ray Train, Ray AIR checkpoints, and evaluation workloads. However, the

final local Jenkins deployment uses FastAPI as the serving backend:

```
python -m madewithml.serve --run_id <run_id> --backend fastapi
```

This decision was made because Ray Serve's HTTP proxy was unstable in the local Docker/Jenkins setup. FastAPI provides a reliable HTTP layer for the demonstration while preserving Ray for the ML workflow. Ray Serve remains available through:

```
python -m madewithml.serve --run_id <run_id> --backend ray-serve
```

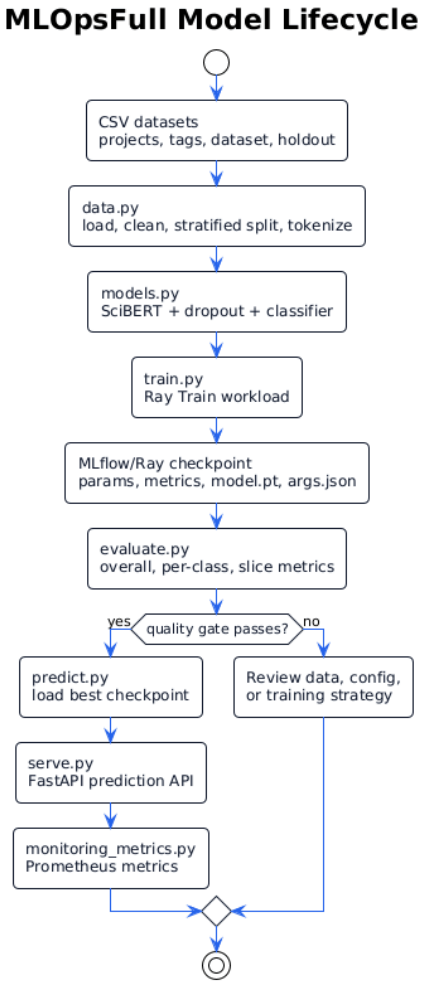

Figure 9: PlantUML diagram of the model lifecycle from CSV data to monitored API.

Model Lifecycle The serving process loads the best checkpoint for the Jenkins-generated `run_id`. The checkpoint includes the model weights, model arguments, and metadata such as the class-to-index mapping. This allows the API to preprocess incoming requests in the same way the training pipeline did.

Prediction API The main endpoint is:

```
POST http://localhost:8000/predict/
```

Example payload:

```
{
  "title": "Text classification with transformers",
  "description": "A project using BERT for NLP classification"
```

```
}
```

The API returns prediction results and monitoring information. Low-confidence predictions are mapped to the fallback class `other`, which helps avoid pretending that the model is confident when it is not.

2.5 Part 5: Monitoring

2.5.1 Course Concept

The monitoring course material explains that the real work begins after deployment. Traditional software is usually deterministic: if the code is correct, the same input should produce the same intended behavior. Machine learning systems are probabilistic and depend on the data distribution. A model can degrade when production data becomes different from training data.

The course separates monitoring into several layers:

- **System health**: latency, throughput, error rates, CPU, GPU, and memory.
- **Model performance**: accuracy, precision, recall, F1, and business metrics.
- **Outliers**: individual unusual inputs.
- **Drift**: distribution-level changes between reference data and production windows.
- **Response workflow**: alert, inspect, and act.

The project implements the educational foundation for these ideas through Prometheus metrics, Grafana dashboards, model behavior tracking, and simple drift helper functions.

2.5.2 Monitoring Architecture

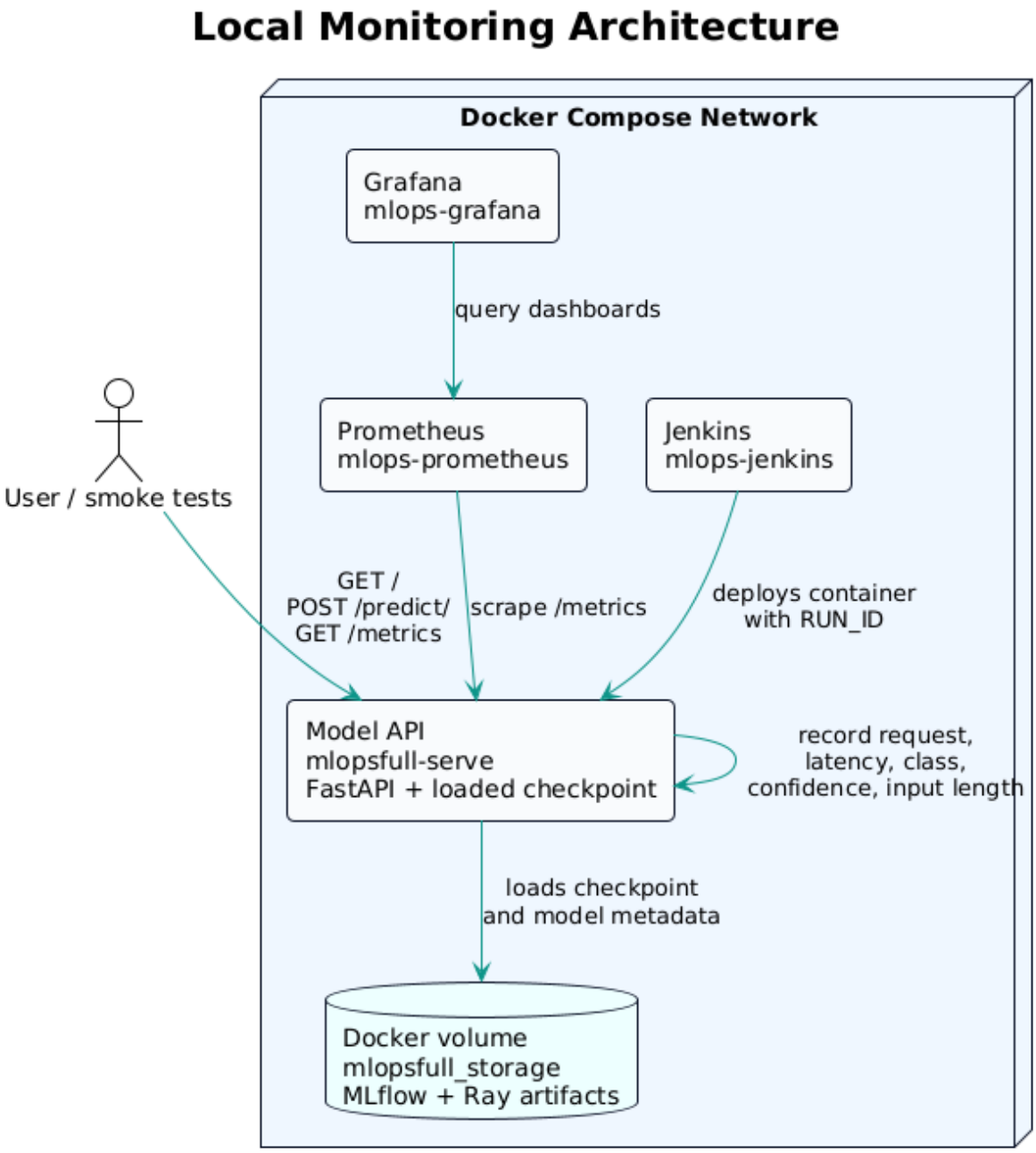

Figure 10: PlantUML diagram of the local monitoring architecture.

The local architecture contains four main services:

Service	URL	Role
Jenkins	`http://localhost:8080`	Builds, tests, trains, evaluates, pushes, and deploys.
Model API	`http://localhost:8000`	Serves predictions and exposes metrics.
Prometheus	`http://localhost:9090`	Scrapes `/metrics` from the model API.

Grafana	`http://localhost:3000`	Visualizes API and model behavior metrics.

2.5.3 Metrics Endpoint

The model service exposes:

```
GET /metrics
```

This endpoint is implemented in `serve.py` using `monitoring_metrics.render_metrics()`. It returns Prometheus-format metrics, which Prometheus scrapes from the Docker network target:

```
mlopsfull-serve:8000/metrics
```

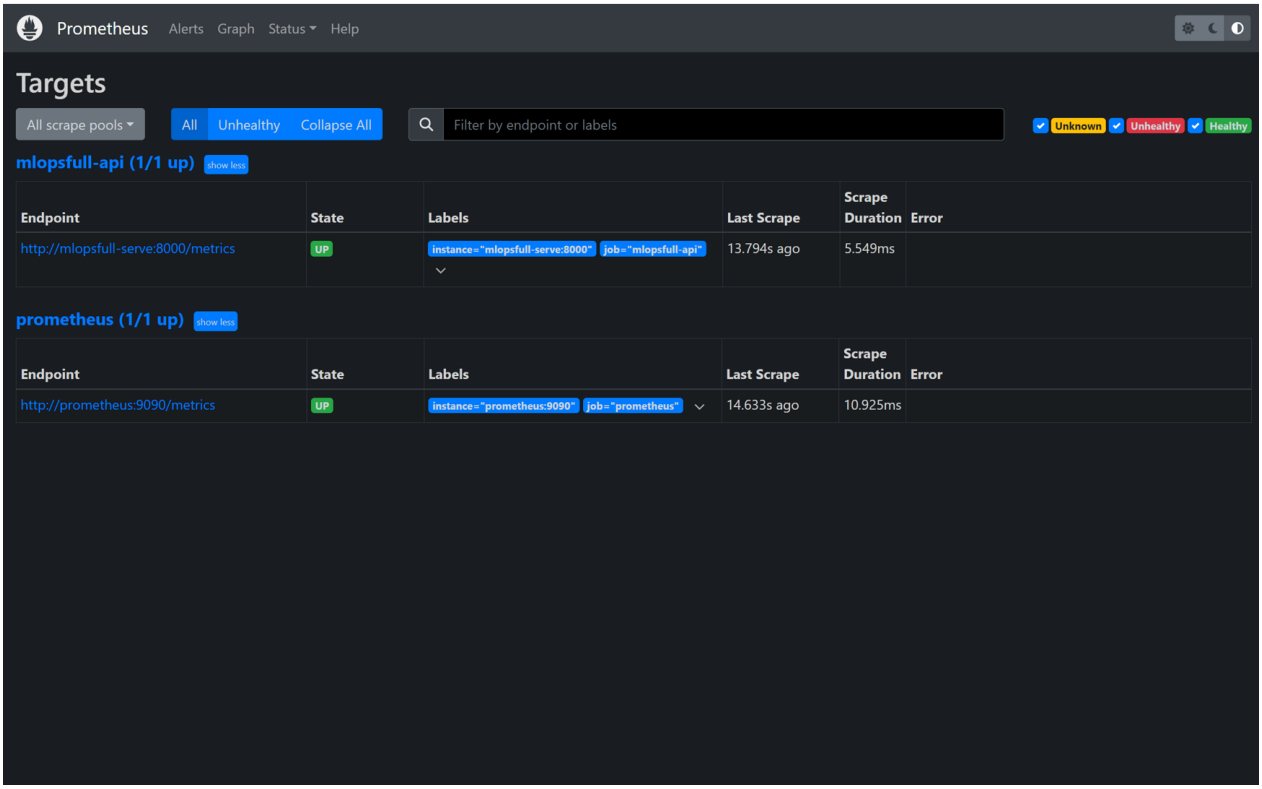

Figure 11: Prometheus target evidence showing the model API metrics endpoint in the `UP` state.

2.5.4 Metrics Collected

Metric	Meaning
`mlopsfull_requests_total`	Number of HTTP requests by endpoint, method, and status.
`mlopsfull_request_latency_seconds`	Latency distribution for prediction and evaluation endpoints.
`mlopsfull_predictions_total`	Number of predictions emitted per predicted class.
`mlopsfull_prediction_confidence`	Histogram of prediction confidence values.
`mlopsfull_input_text_length_tokens`	Histogram of input title and description lengths.
`mlopsfull_other_prediction_rate`	Latest rate of predictions mapped to the fallback `other` class.

`mlopsfull_input_validation_failures_total`	Count of empty title or description validation failures.

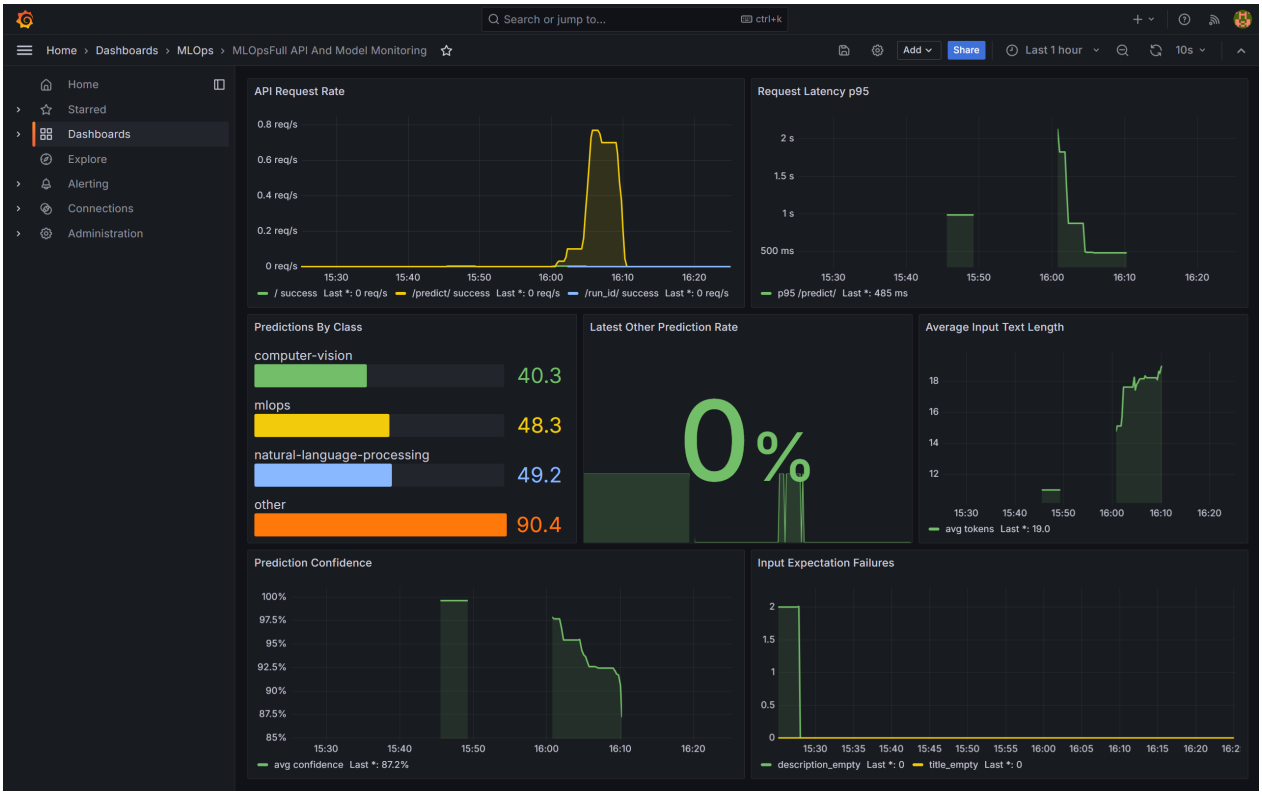

Figure 12: Grafana dashboard evidence visualizing API request rate, latency, predictions, confidence, and input validation metrics.

2.5.5 Monitoring Helper Logic

The file `monitoring.py` contains logic for:

- calculating request text length;
- validating empty title and description fields;
- extracting prediction confidence;
- summarizing prediction outputs;
- detecting text length drift;
- detecting class distribution drift.

These helpers reflect the monitoring PDF's distinction between health checks and model behavior checks. Health checks answer whether the API is running. Model behavior checks answer whether the model is behaving usefully.

2.5.6 Drift Foundation

The project includes two lightweight drift checks:

```
detect_text_length_drift(reference_lengths, current_lengths)
detect_class_distribution_drift(reference_classes, current_classes)
```

The first compares average text length between a reference window and a current window. The second compares class distributions using total variation distance. These are intentionally simple and educational. In production, this could be extended with tools such as EvidentlyAI, WhyLogs, Alibi Detect, or managed platforms.

2.5.7 Traffic Generation Result

A traffic script sends repeated prediction requests to populate Prometheus and Grafana:

```
python scripts/send_monitoring_traffic.py --url http://localhost:8000/predict/ --requests 200
```

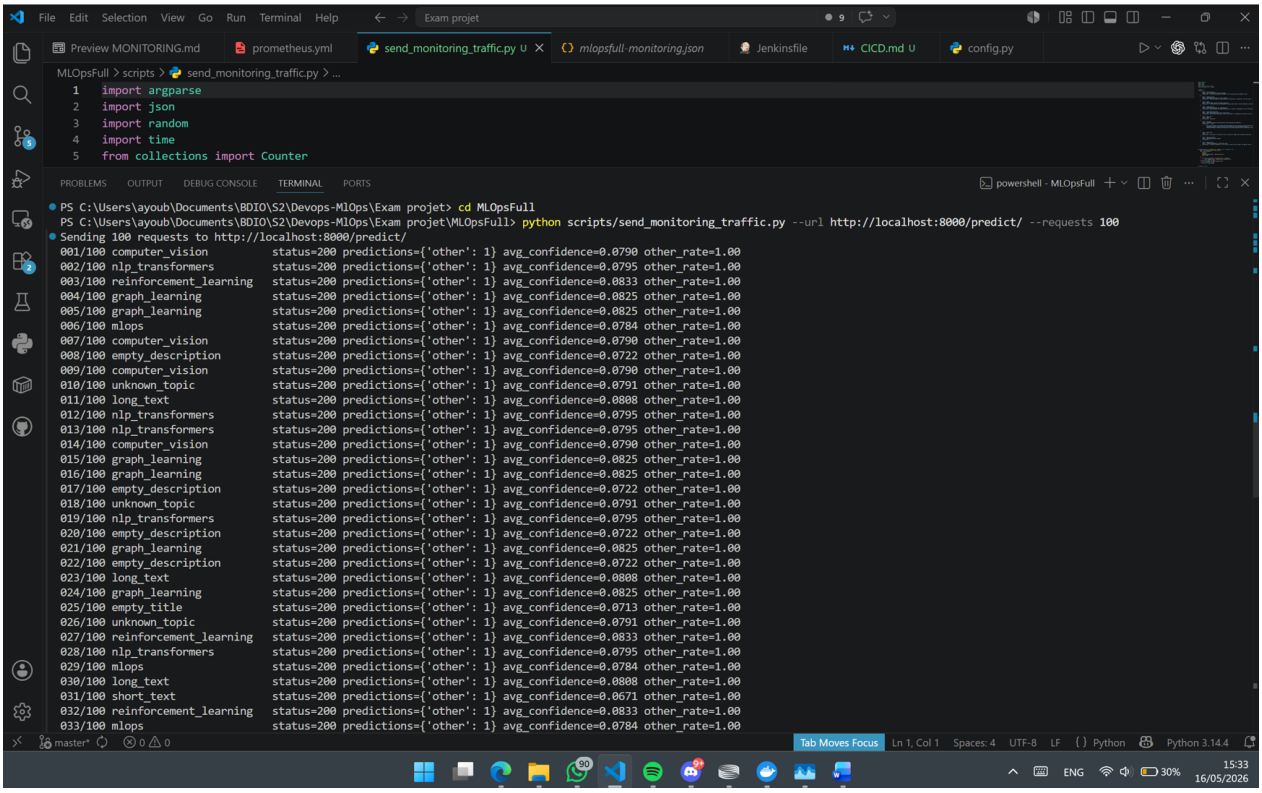

Figure 13: Traffic generation evidence showing repeated prediction requests sent to the deployed API.

The monitoring traffic run produced:

```
Total requests: 200
HTTP 200 responses: 200
HTTP errors: 0
```

The observed model behavior was consistent with the evaluation result. The deployed model produced successful predictions across the target classes, including `natural-language-processing`, `computer-vision`, `mlops`, and `other`. Prometheus recorded `230` prediction requests in total, with an average monitored confidence of approximately `0.9285`. This shows that monitoring captures both API health and model behavior after deployment.

2.5.8 Alert, Inspect, Act

The course monitoring workflow is:

```
alert -> inspect -> act
```

The current project implements the measurement layer needed before alerting. A production extension could add Prometheus alert rules such as:

- alert if request errors increase;
- alert if prediction latency crosses a threshold;
- alert if `other` prediction rate remains high;
- alert if average confidence drops below a threshold;
- alert if text length or class distribution drift is detected.

After alerting, the team should inspect metadata, thresholds, logs, reference data, current data, and model outputs before acting. Possible actions include checking data expectations, confirming that the schema did not change, retraining the model, updating the reference window, or investigating whether outliers are valid new production examples.

3 Limitations And Future Improvements

3.1 Current Limitations

- The monitoring setup is local and Docker-based.
- The training dataset is small and stored locally for this educational project.
- The deployment gate is configurable and currently enforces a stricter weighted F1 threshold.
- Drift helpers exist, but they are not yet scheduled as recurring production jobs.
- No Alertmanager, Slack, email, or PageDuty alert routing is configured.
- Ray Serve remains available, but FastAPI was selected for stable local deployment.

3.2 Future Improvements

- Add more labeled data and scheduled retraining for broader generalization.
- Calibrate deployment thresholds as production requirements evolve.
- Add Prometheus alert rules and Alertmanager notifications.
- Run drift checks on scheduled windows or event streams.
- Store prediction events for later audit and retraining analysis.
- Add DVC or object storage for larger dataset versioning.
- Deploy to Kubernetes or a managed ML platform for a more production-like environment.

4 Conclusion

This project implements an end-to-end MLOps workflow for a text classification model. The first foundation is software structure: notebook logic is separated into reusable Python modules for data processing, modeling, training, tuning, evaluation, prediction, serving, and monitoring. The second foundation is reproducibility: Git, dependency files, MLflow, Ray checkpoints, deterministic utilities, and archived artifacts make the workflow traceable.

The CI/CD implementation extends the project from a local ML codebase into an automated delivery workflow. Jenkins builds the Docker image, runs code checks, trains the model, evaluates it, gates deployment with weighted F1, publishes the image, deploys the API, and archives evidence. The monitoring implementation then connects the deployed API to Prometheus and Grafana, exposing both system behavior and model behavior metrics.

The final result is not only a trained model, but a working MLOps system: code changes can trigger automated validation, model training can produce tracked artifacts, deployment is guarded by

evaluation, and the running service can be observed after release. This is the central lesson of the project: machine learning engineering is not just about producing a model, but about building the operational workflow that keeps the model reliable, traceable, and improvable.